

Recursive Functions on Lists in Isabelle/HOL

```
theory List_Recursion
imports Main
begin
```

Lists are some of the most basic structures in computation. You can use them to represent data structures with a number of elements and that are ordered in a sequence. In functional programming, such lists can be computationally processed using recursion. The following example shows how to recursively define functions that add up all the numbers in a list of natural numbers.

```
fun sum::“nat list  $\Rightarrow$  nat” where
  “sum [] = 0”
| “sum (x # xs) = x + sum xs”

value “sum [1,2]”
```

```
thm sum.simps
```

Note the output for the given list and the theorems corresponding to the defining equations.

A function that appends two lists to form one list containing all elements of the given two lists

```
fun Append::“'a list  $\Rightarrow$  'a list  $\Rightarrow$  'a list” where
  “Append [] ys = ys”
| “Append (x # xs) ys = x # (Append xs ys)”

thm Append.simps
```

```
value “Append ([1,2]::nat list) [3,4]”
```

Three different functions to compute the largest number in a list using different Isabelle/HOL constructs: *if-then-else*, *max*, which is a predefined function, and *let-in*.

```
fun Max_list::“nat list  $\Rightarrow$  nat” where
  “Max_list [] = 0”
| “Max_list (x # xs) = (if (x  $\geq$  Max_list xs) then x else (Max_list xs))”
```

```
fun Max_list'::“nat list  $\Rightarrow$  nat” where
  “Max_list' [] = 0”
| “Max_list' (x # xs) = max x (Max_list' xs)”
```

```
fun Max_list''::“nat list  $\Rightarrow$  nat” where
  “Max_list'' [] = 0”
| “Max_list'' (x # xs) = (let M = Max_list xs in (if x  $\geq$  M then x else M))”
```

```
value “Max_list [4,1,4,5,2,3]”
```

thm *Max_list.simps*

end